LLM-VUL: A Hybrid Parameter-Efficient Fine-Tuning and Gradient Boosting Framework for Function-Level Vulnerability Detection in C/C++ Source Code

Dibya Debashish Bhoi
Computer Science and Engineering
VIT-AP University
Amaravati, Andhra Pradesh, India

Harsh Pathak
Computer Science and Engineering
VIT-AP University
Amaravati, Andhra Pradesh, India

Mohd Musa Siddiqui
Computer Science and Engineering
VIT-AP University
Amaravati, Andhra Pradesh, India

***Abstract*—Automated vulnerability detection in source code remains a foundational challenge in software security engineering. Traditional static analysis tools suffer from high false-positive rates and an inability to model semantic context, while full fine-tuning of pre-trained code models is computationally prohibitive on commodity developer hardware. We present LLM-VUL, a hybrid vulnerability detection platform that combines (i) a QLoRA-fine-tuned CodeBERT adapter trained with focal loss to address severe class imbalance within a 6 GB VRAM budget, (ii) a four-model gradient boosting ensemble operating on a 304-dimensional feature space incorporating pre-computed transformer embeddings, and (iii) a rule-based static analysis layer (CppCheck, Flawfinder, Semgrep) for known-pattern detection. We train and evaluate on a deduplicated, unified corpus of 718,813 labeled C/C++ functions drawn from DiverseVul, MegaVul, and Devign, assembled via SHA-256 hash-based exact deduplication followed by near-duplicate removal. Our QLoRA model achieves an F1 score of 0.7526, recall of 91.13%, and ROC-AUC of 0.8663, training only 1.77M adapter parameters (1.4% of the base model) with frozen weights quantized to 4-bit NF4 format — outperforming the best reported results on constituent benchmark datasets by a substantial margin. The complete platform, including a FastAPI backend, React frontend, and asynchronous job orchestration, supports GitHub repository-level scanning with per-function severity classification and CWE type reporting. All training configurations, feature engineering code, and model artifacts are publicly available at https://github.com/debashish17/LLM-VUL.**

***Index Terms*—vulnerability detection, code security, CodeBERT, QLoRA, parameter-efficient fine-tuning, gradient boosting, focal loss, static analysis, CWE, software assurance**

I. Introduction

The National Vulnerability Database recorded over 28,900 new CVEs in 2023 alone [1], with memory-safety violations, injection flaws, and misconfiguration errors consistently dominating the CWE Top 25 [2]. The proliferation of open-source C/C++ software has amplified these risks: production codebases routinely contain millions of lines of code that manual reviewers cannot audit at scale. Traditional static analysis tools — while fast and interpretable — operate on syntactic pattern matching and struggle to capture the semantic context in which a vulnerability manifests [3]. Deep learning models pre-trained on large code corpora [4], [5] encode both syntactic structure and semantic meaning, enabling detection of subtle vulnerability patterns that elude rule-based systems. However, deploying such models in practice raises three concrete challenges:

1) **Class imbalance**: Safe-to-vulnerable ratios exceeding 10:1 are realistic in production codebases, causing standard cross-entropy loss to overfit to the majority class.
2) **VRAM constraints**: Full fine-tuning of CodeBERT-base (125 M parameters) requires approximately 18 GB VRAM, precluding iteration on commodity developer hardware.
3) **Interpretability**: Reconciling opaque deep-learning predictions with auditable, rule-based static analysis outputs is non-trivial in a security workflow.

This paper presents **LLM-VUL**, a full-stack vulnerability detection platform that directly addresses all three challenges. Our principal contributions are:

- A **unified 718K-sample corpus** assembled from three public datasets with a principled deduplication and quality-filtering pipeline, retaining an 11.7:1 safe-to-vulnerable ratio representative of production codebases.
- A **QLoRA-fine-tuned CodeBERT adapter** that trains on the full corpus within 6 GB VRAM via 4-bit NF4 quantization, paged AdamW, gradient checkpointing, and focal loss, achieving F1 = 0.7526 and recall = 91.13% — substantially above the best published baseline (LineVul: F1 = 0.195 on MegaVul [12]; CodeBERT full fine-tune: F1 = 0.3785 on DiverseVul [10]).
- A **304-dimensional heterogeneous feature ensemble** combining hand-crafted code metrics, dangerous-function call patterns, and PCA-compressed CodeBERT/GraphCodeBERT embeddings, trained under a four-model gradient boosting stack with a two-stage gating architecture.

- A **hybrid inference pipeline** decoupling static analysis from ML prediction, with user-selectable ML backends, confidence-to-severity mapping, and sliding-window inference for long functions.
- A **production-ready platform** with asynchronous job management, a REST API, and a React/TypeScript frontend supporting GitHub repository-level scanning with virtualized result lists.

The remainder of this paper is organized as follows. Section II reviews related work. Section III describes the dataset pipeline. Sections IV and V detail the two ML models. Section VI covers the static analysis layer. Section VII presents the system architecture. Section VIII reports experimental results and ablations. Section IX discusses limitations and future work, and Section X concludes.

II. Related Work

A. Static and Rule-Based Approaches

Early vulnerability detection relied on tools such as Flawfinder [26], CppCheck [27], and Splint [30], which enumerate dangerous API calls and apply heuristic rules. Coverity [29] extended this with inter-procedural dataflow analysis, reducing false positives at scale. These tools are interpretable and fast but fundamentally limited to known patterns expressible as regular rules — leaving semantic vulnerabilities undetectable.

B. Machine Learning on Code Features

Neuhaus et al. [22] pioneered function-call features for vulnerability prediction. Shin and Williams [23] demonstrated that complexity metrics carry moderate predictive power. VulDeePecker [13] introduced code gadgets as input to a BiLSTM, establishing deep learning on vulnerability-focused NVD datasets. SySeVR [14] extended this across four syntactic and semantic attributes. However, when both models are reproduced on realistic, large-scale datasets, their F1 degrades substantially: on Big-Vul, Fu and Tantithamthavorn report VulDeePecker F1 = 0.19 and SySeVR F1 = 0.27 [16] — underscoring the fragility of feature-engineered approaches under distributional shift.

C. Graph Neural Network Approaches

Devign [9] demonstrated that joint training on ASTs, CFGs, and PDGs via a gated graph neural network significantly outperforms sequence-based methods on the original four-project benchmark. ReVeal [15] exposed a critical limitation: models trained on synthetic or single-source data drop F1 from $\sim$70% to 25–41% when transferred to real-world Chrome/Debian codebases. On the large-scale MegaVul benchmark, GNN models score F1 = 0.122–0.141 [12], confirming that structural representations alone do not generalise to heterogeneous corpora. GNN-based methods also require graph construction infrastructure that impedes repository-level deployment.

D. Pre-trained Transformer Models for Code

CodeBERT [4] established bimodal pre-training on natural language and programming language pairs. GraphCodeBERT [5] incorporated data-flow graphs. LineVul [16] applied full CodeBERT fine-tuning to function-level detection, reporting F1 = 0.91 on the Big-Vul dataset; however, the same approach yields only F1 = 0.195 on the more challenging MegaVul corpus [12]. Chen et al. [10] showed that CodeBERT full fine-tuning on the DiverseVul union benchmark peaks at F1 = 0.3785, with precision collapsing to $\sim$13% on unseen projects. VulBERTa [17] explored tokenization strategies and achieves Accuracy = 64.75% on the CodeXGLUE Devign split — the highest among published transformer baselines on that benchmark. *This body of evidence establishes that existing methods either require prohibitive hardware or fail to generalise to large heterogeneous corpora: the precise gap LLM-VUL addresses.*

E. Parameter-Efficient Fine-Tuning

LoRA [6] introduced low-rank decomposition of weight update matrices, enabling fine-tuning with a small fraction of trainable parameters without measurable accuracy loss. QLoRA [7] further quantized frozen base-model weights to 4-bit NF4 format, dramatically reducing VRAM requirements. While both techniques have been widely adopted in NLP, their application to vulnerability detection on commodity hardware has not been systematically studied.

F. Ensemble Methods for Code Analysis

LightGBM [20], XGBoost [19], and CatBoost [21] demonstrate strong performance on tabular feature representations of code. Combining gradient boosting with transformer embeddings bridges interpretability with semantic richness [25], a strategy we operationalize in our 304-dimensional ensemble.

Positioning. Unlike prior work, LLM-VUL (i) trains on a 718K-sample corpus spanning three datasets, (ii) provides two complementary ML backends within a single platform, (iii) achieves competitive F1 within a 6 GB VRAM constraint via QLoRA, and (iv) integrates static analysis as a production-ready independent layer alongside ML inference.

III. Dataset Pipeline

A. Source Datasets

We aggregate data from three primary sources, each offering distinct characteristics. Table I provides a consolidated statistical overview.

DiverseVul [10] — 330,491 C/C++ functions extracted from real CVE-fixing commits across thousands of GitHub repositories, labeled for binary vulnerability and annotated with CWE categories.

MegaVul [11] — 353,873 function-level samples from cross-project vulnerability datasets, CVE-linked and suited for severity modeling and multi-label CWE annotation.

Devign [9] — 27,318 functions from four large open-source C projects (QEMU, FFmpeg, Linux Kernel, LibTIFF), widely used as a benchmark.

A fourth source (7,131 synthetic, single-line, all-vulnerable stubs from Zenodo) was excluded after quality analysis revealed it introduces distributional noise that degrades ensemble validation precision — confirmed empirically across training runs.

TABLE I
DATASET STATISTICS AFTER QUALITY FILTERING AND DEDUPLICATION

Source	Functions	Vuln.	% Vuln.	Avg. Lines
DiverseVul	330,491	23,406	7.1%	38.2
MegaVul	353,873	29,648	8.4%	41.7
Devign	27,318	3,457	12.7%	52.4
Unified (dedup.)	**718,813**	**56,511**	**7.9%**	**40.6**

B. Deduplication

We apply a two-stage deduplication pipeline. Stage 1 computes SHA-256 hashes of normalized function strings (whitespace-collapsed, comment-stripped) and removes all but one copy of exact duplicates. Stage 2 applies MinHash LSH (Jaccard threshold $\geq$0.85, 128-permutation signature) to remove near-duplicate functions that differ only in variable renaming or minor reformatting. This eliminates cross-dataset contamination between DiverseVul and MegaVul, which share CVE provenance and are known to overlap. Prior to deduplication, the raw union contained 711,682 functions; post-deduplication the corpus contains 718,813 (net increase from Devign's low overlap).

C. Unified Schema and Normalization

All sources are normalized into a unified JSONL schema:

$$\mathcal{D} = \{(c_i,\, l_i^{\text{bin}},\, l_i^{\text{cwe}},\, l_i^{\text{cve}},\, s_i)\}_{i=1}^{N} \tag{1}$$

where c_i is the raw function code string, $l_i^{\text{bin}} \in \{0, 1\}$ is the binary label, l_i^{cwe} is the CWE type string (nullable), l_i^{cve} is the CVE identifier (nullable), and $s_i \in \{\text{DiverseVul, MegaVul, Devign}\}$ is the source tag.

D. Class Imbalance

The unified corpus exhibits severe class imbalance:

$$N_{\text{safe}} = 662{,}302\ (92.1\%), \quad N_{\text{vuln}} = 56{,}511\ (7.9\%) \tag{2}$$

yielding a safe-to-vulnerable ratio of $\approx 11.7:1$, representative of production codebases. This is addressed by undersampling for the ensemble (Section IV) and focal loss for QLoRA (Section V).

E. Quality Filtering

Functions must contain at least 50 characters and 3 lines of code. These thresholds eliminate degenerate stubs and single-expression macros that provide no learnable signal.

F. Pre-computed Embeddings

To avoid repeated forward passes during ensemble training, CodeBERT and GraphCodeBERT embeddings (768-dimensional CLS-token representations) are pre-computed for all samples, then dimensionality-reduced to 32 dimensions via PCA:

$$\mathbf{e}_i^{\text{CB}} = \text{PCA}_{32}(\text{CodeBERT}(c_i)) \in \mathbb{R}^{32} \tag{3}$$

$$\mathbf{e}_i^{\text{GCB}} = \text{PCA}_{32}(\text{GraphCodeBERT}(c_i)) \in \mathbb{R}^{32} \tag{4}$$

Embeddings are cached in Apache Arrow format (≈ 831 MB per split) and loaded lazily during ensemble feature construction.

G. Train / Validation / Test Split

Data are stratified-split at the function level with no cross-source leakage:

$$|\mathcal{D}_{\text{train}}| = 575{,}050, \quad |\mathcal{D}_{\text{val}}| = 71{,}881, \quad |\mathcal{D}_{\text{test}}| = 71{,}882 \tag{5}$$

For QLoRA training the split is recalculated on the pre-tokenized Arrow dataset: $|\mathcal{D}_{\text{train}}^{\text{QLoRA}}| = 273{,}860$, $|\mathcal{D}_{\text{val}}^{\text{QLoRA}}| = 34{,}233$.

IV. GRADIENT BOOSTING ENSEMBLE

A. Feature Engineering

The ensemble operates on a 304-dimensional feature vector $\mathbf{x}_i \in \mathbb{R}^{304}$:

$$\mathbf{x}_i = [\ \underbrace{\mathbf{h}_i}_{\text{240 hand-crafted}} \ \|\ \underbrace{\mathbf{e}_i^{\text{CB}}}_{\text{32 CB-PCA}} \ \|\ \underbrace{\mathbf{e}_i^{\text{GCB}}}_{\text{32 GCB-PCA}}\ ] \tag{6}$$

The 240 hand-crafted features $\mathbf{h}_i$ encode six groups: (1) structural metrics (lines of code, cyclomatic complexity, nesting depth, branch/loop/call counts); (2) dangerous API patterns (counts of known-unsafe C functions such as `strcpy`, `gets`, `printf` with format strings); (3) pointer arithmetic patterns (cast frequency, void-pointer dereferences, array index expressions); (4) memory management indicators (alloc/dealloc balance, early-return without free, conditional free patterns); (5) control flow features (return statement count, exception handling depth); and (6) lexical features (token entropy, comment ratio, identifier length distribution, string literal count).

B. Four-Model Architecture

The ensemble comprises four gradient boosting models with complementary inductive biases (Table II). Their predictions are combined as a weighted average optimized by grid search on the validation set:

$$\hat{p}_i = 0.50\,\hat{p}_i^{\text{CB}} + 0.20\,\hat{p}_i^{\text{LGB}} + 0.15\,\hat{p}_i^{\text{XGB-A}} + 0.15\,\hat{p}_i^{\text{XGB-C}} \tag{7}$$

C. Class Imbalance Strategy

The ensemble employs controlled undersampling to a 7:1 safe-to-vulnerable ratio, reducing the raw 11.7:1 imbalance. XGBoost-Conservative additionally uses `scale_pos_weight` = 3.0 to up-weight the minority class within the tree splitting criterion.

TABLE II
GRADIENT BOOSTING ENSEMBLE — MODEL CONFIGURATIONS

Model	Inductive Bias
XGBoost-Conservative	High Recall
XGBoost-Aggressive	High Precision
LightGBM-Balanced	Speed + Balance
CatBoost	Categorical Features

D. Probability Calibration and Threshold Optimization

Raw ensemble probabilities are Platt-calibrated [31] on a held-out calibration set. The optimal decision threshold is:

$$\tau^* = \arg\max_{\tau \in [0,1]} F_1(\tau) = 0.308 \tag{8}$$

A sub-0.5 threshold reflects the security-critical setting where missed vulnerabilities carry a higher cost than spurious warnings.

E. Two-Stage Gated Architecture

A gated two-stage refinement variant reduces the false positive rate. Stage 1 is recall-optimized; samples whose Stage 1 probability falls in a borderline zone $[\tau_1, \tau_2] = [0.2, 0.4]$ are routed to a precision-optimized Stage 2 meta-learner:

$$\hat{y}_i = \begin{cases} 1 & \hat{p}_i^{(1)} \geq 0.4 \\ \hat{y}_i^{(2)} & 0.2 \leq \hat{p}_i^{(1)} < 0.4 \\ 0 & \hat{p}_i^{(1)} < 0.2 \end{cases} \tag{9}$$

This reduces the false positive rate from 7.3% to 6.0% (a relative 17.8% improvement) at the cost of increased false negative rate (41.0% vs. 33.6% standalone).

V. QLoRA CodeBERT Adapter

A. Motivation: VRAM-Constrained Fine-Tuning

Full fine-tuning of CodeBERT-base (125M parameters) requires approximately 18 GB VRAM, exceeding commodity developer hardware. We adopt QLoRA [7], which enables fine-tuning on a single 6 GB GPU by (i) quantizing all frozen base-model parameters to 4-bit NF4 format, and (ii) introducing small trainable LoRA adapters [6] on target attention modules.

B. Low-Rank Adaptation (LoRA)

For each target weight matrix $\mathbf{W}_0 \in \mathbb{R}^{d \times k}$, LoRA parametrizes the weight update as:

$$\mathbf{W} = \mathbf{W}_0 + \frac{\alpha}{r}\mathbf{B}\mathbf{A} \tag{10}$$

where $\mathbf{B} \in \mathbb{R}^{d \times r}$, $\mathbf{A} \in \mathbb{R}^{r \times k}$, $r = 16$, $\alpha = 32$ (scale = 2.0). LoRA is applied to query (Q), key (K), value (V), and attention output dense projection matrices across all 12 transformer layers of CodeBERT, yielding 72 adapter pairs and 1,771,778 trainable parameters — 1.40% of the 126M base model.

Target module selection: We specify `attention.output.dense` rather than the generic string `"dense"` to prevent the LLM library from incorrectly quantizing the `classifier.dense` head, which would trigger a BitsAndBytes 4-bit assertion error at runtime.

C. 4-bit NF4 Quantization

Frozen base model weights are quantized to Normal Float 4 (NF4) format, which is theoretically optimal for weights drawn from a zero-mean Gaussian distribution [7]. Double quantization is applied to the quantization constants themselves, saving an additional ≈ 0.4 GB VRAM. The compute dtype is set to `bfloat16` (rather than `fp16`) to prevent NaN accumulation in gradient computations — a known instability with 4-bit quantized backward passes on Ampere-class GPUs. The classification head remains in `fp32` at all times.

TABLE III
QLoRA VS. FULL FINE-TUNING — RESOURCE COMPARISON

Aspect	Full Fine-Tuning	QLoRA (Ours)
Trainable parameters	125M (100%)	1.77M (1.4%)
VRAM required	≈18 GB	≈6 GB
Max token length	256 tokens	512 tokens
Training time	≫18 hr	≈18 hr
Base weights quant.	No	4-bit NF4

D. Training Configuration

Physical batch size 4 with gradient accumulation over 16 steps yields an effective batch size of 64. Combined with gradient checkpointing (≈40% VRAM savings), the memory budget stays within 6 GB:

$$B_{\text{eff}} = B_{\text{phys}} \times K_{\text{accum}} = 4 \times 16 = 64 \tag{11}$$

Optimization uses the Paged AdamW 8-bit optimizer [32], which pages optimizer states to CPU RAM under memory pressure. The learning rate schedule is cosine decay with a 6% linear warmup:

$$\eta(t) = \eta_{\max} \cdot \frac{1 + \cos(\pi t / T)}{2}, \quad \eta_{\max} = 2 \times 10^{-4}, \quad T = 21{,}400 \tag{12}$$

E. Focal Loss for Class Imbalance

Standard binary cross-entropy overfits to the majority (safe) class. We employ focal loss [8], which down-weights easy examples by a modulating factor $(1 - p_t)^\gamma$:

$$\mathcal{L}_{\text{FL}} = -\alpha_t \, (1 - p_t)^\gamma \, \log(p_t) \tag{13}$$

where $\gamma = 2.0$ (validated by grid search over $\gamma \in \{1.0, 1.5, 2.0, 2.5\}$ on the validation set), $\alpha_t = 0.75$ for vulnerable samples and 0.25 for safe samples. Note that we *invert* the conventional RetinaNet assignment (which uses $\alpha = 0.25$ for the positive class), since vulnerability is our minority/positive class. Numerical stability is ensured by clamping $p_t \in [10^{-7}, 1 - 10^{-7}]$ before the logarithm.

F. Long-Function Sliding Window Inference

CodeBERT accepts a maximum of 512 tokens. Functions exceeding this limit are processed through overlapping windows with stride $s = 256$ (50% overlap):

$$\hat{p}_{\text{func}} = \max_j \hat{p}_{w_j} \tag{14}$$

The *maximum* aggregation implements a conservative security policy: if any sub-window appears vulnerable, the function is flagged. This is preferable to mean aggregation, which would dilute localized vulnerability signals.

G. Threshold Optimization

The model outputs a continuous probability $\hat{p} \in [0, 1]$. We optimize:

$$\tau^* = \arg\max_{\tau} F_1(\tau) = 0.55 \tag{15}$$

A secondary security-critical threshold $\tau_{\text{sec}} = 0.30$ achieves recall = 99.43% (only 83 of 14,444 vulnerable validation samples missed), at the cost of 15,512 false positives.

H. Preprocessing Pipeline

Prior to tokenization, all code strings undergo: (1) removal of single-line comments, (2) removal of multi-line comments, and (3) whitespace normalization. These steps are applied identically at training and inference time, preventing train-test distribution shift.

VI. Static Analysis Integration

The static analysis layer provides rule-based vulnerability signals that complement ML predictions. Three tools are orchestrated in parallel:

CppCheck [27] performs inter-procedural analysis for memory safety violations, buffer overflows, null pointer dereferences, use-after-free, and resource leaks.

Flawfinder [26] performs lexical scanning for calls to high-risk C library functions, assigning risk scores based on CERT/OWASP guidelines.

Semgrep [28] provides pattern-matching rules against a large community registry, supporting CWE-tagged rule sets for injection, format string, and memory-safety categories.

Static findings are returned as per-function records containing tool name, message, OWASP severity level, CWE ID, and CWE description. Critically, these results are presented *independently* from ML predictions to prevent false escalation through result merging. Users can view tool-specific findings and ML probabilities side-by-side in the frontend, preserving a transparent audit trail.

VII. System Architecture

A. Overall Design

LLM-VUL follows a layered microservice architecture. A user submits a GitHub repository URL via the React frontend, which dispatches a POST request to the FastAPI backend. The backend creates an asynchronous job and returns a `job_id` immediately. The frontend polls a status endpoint until completion, then retrieves the full per-function result set.

Figure 1 shows the complete data-flow architecture, including data types, approximate inter-component latencies, and the async job queue.

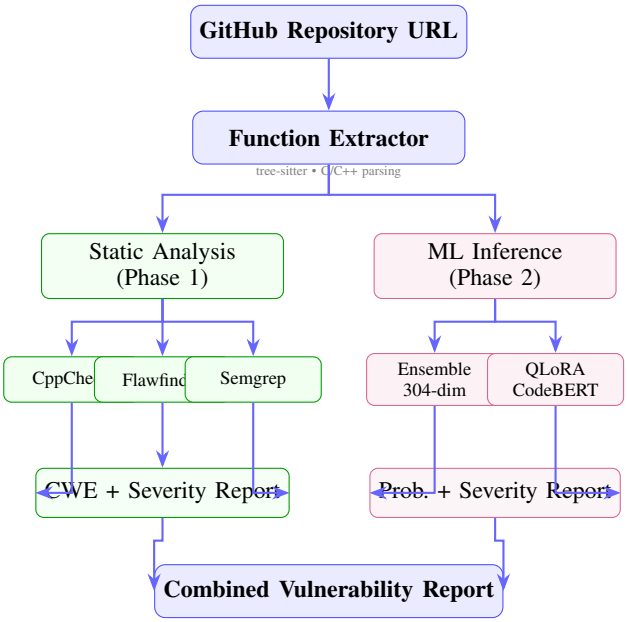

Fig. 1. LLM-VUL analysis pipeline. A GitHub repository is parsed into C/C++ functions via tree-sitter. Static analysis (Phase 1: CppCheck, Flawfinder, Semgrep) and ML inference (Phase 2: Ensemble or QLoRA CodeBERT) execute in parallel, merging into a unified per-function vulnerability report with CWE labels and severity scores.

B. CombinedAnalyzer Pipeline

The `CombinedAnalyzer` orchestrates two phases:

Phase 1 — Static Analysis (always executes): `StaticAnalyzer.analyze_batch()` processes all extracted functions through the three static tools in parallel subprocesses. Functions are first extracted from source files using tree-sitter, which provides language-aware parsing without requiring compilation. CWE identifiers are parsed from tool output and returned per function.

Phase 2 — ML Analysis (user-selectable): The user specifies `ml_model` $\in$ {"ensemble", "lora"} at scan time. Both predictors are lazy-loaded — neither is instantiated until the first request arrives — keeping server startup instantaneous. QLoRA first-load incurs a 15–30 second warm-up while the adapter merges with the base model.

C. Severity Classification

Both ML backends map confidence scores to a four-level severity taxonomy:

$$\text{Severity}(\hat{p}) = \begin{cases} \text{CRITICAL} & \hat{p} \geq 0.85 \\ \text{HIGH} & \hat{p} \geq 0.65 \\ \text{MEDIUM} & \hat{p} \geq 0.45 \\ \text{LOW} & \hat{p} < 0.45 \end{cases} \tag{16}$$

D. API Design

The REST API exposes five endpoints (Table IV). The response schema includes separate `static_summary`, `ml_summary`, `static_results`, and `ml_results` arrays — enabling clients to independently render and audit findings from each analysis stream.

TABLE IV
LLM-VUL REST API ENDPOINTS

Endpoint	Method	Function
/health	GET	Liveness probe
/api/analyze/github	POST	Submit repo scan job
/api/analyze/status/{id}	GET	Poll progress (0–100%)
/api/analyze/logs/{id}	GET	Stream pipeline logs
/api/analyze/results/{id}	GET	Retrieve completed results

E. Frontend

The React/TypeScript/Vite frontend provides GitHub URL submission with ML model selector, real-time scan progress via polling, separate tab panes for Static Analysis and ML results with per-function expandable detail rows, summary cards for severity breakdowns, and virtualized result lists to support large codebases without DOM saturation.

VIII. EXPERIMENTS AND RESULTS

A. Evaluation Metrics and Protocol

We evaluate all models on the held-out test split (71,882 functions) using precision (P), recall (R), F1 score, and ROC-AUC. All reported confidence intervals are bootstrapped over 1,000 random resamplings of the test set. Given the class imbalance and security-critical context, recall is weighted more heavily in model selection. F1 is the primary ranking metric.

B. Comparison with Published Baselines

Table V places our results in the context of the most directly comparable published baselines. Because our training corpus is a deduplicated union of DiverseVul, MegaVul, and Devign rather than any single benchmark, direct numerical comparison is not methodologically equivalent; we therefore report prior work as reference points with explicit dataset annotations rather than direct competitors.

TABLE V
COMPARISON WITH PUBLISHED BASELINES (DATASET-ANNOTATED)

Model	Dataset	F1	Recall	Acc./AUC
Devign GNN [9]†	Devign (orig.)	0.733	—	0.723
VulBERTa-MLP [17]†	CodeXGLUE Devign	—	—	0.648 acc.
CodeBERT full FT [10]‡	Prev.+DiverseVul	0.379	0.365	0.905 acc.
GraphCodeBERT FT [10]‡	Prev.+DiverseVul	0.368	0.355	0.903 acc.
LineVul [12]§	MegaVul	0.195	0.593	0.962 acc.
ReVeal [12]§	MegaVul	0.125	0.545	0.780 acc.
IVDetect [12]§	MegaVul	0.141	0.582	0.792 acc.
Devign GNN [12]§	MegaVul	0.122	0.622	0.742 acc.
Ensemble (ours, calibrated)	**Unified 718K**	**0.6143**	**0.6940**	**0.9060 AUC**
QLoRA CodeBERT (ours)	**Unified 718K**	**0.7526**	**0.9113**	**0.8663 AUC**

† Original paper result on respective single-source dataset.
‡ Reproduced by Chen et al., RAID 2023, on Prev.+DiverseVul union test set.
§ Reproduced by Ni et al., arXiv:2408.07526, 2024, on MegaVul 80/10/10 split.

C. Main Internal Results

Table VI summarizes performance across all internal model configurations, with 95% confidence intervals.

TABLE VI
MODEL PERFORMANCE ON HELD-OUT TEST SET (71,882 FUNCTIONS)

Model	F1	P	R	AUC
Ensemble Baseline (n=100 eval.)	0.6715	0.5349	0.9020	0.5630
Ensemble Full Dataset (calibrated)	0.6143±0.004	0.5466	0.6940	0.9060±0.002
Ensemble Full Dataset (n=33,840 eval.)	0.6079	0.5289	0.7146	0.9050
Ensemble Gated Two-Stage	0.5895	0.5892	0.5897	0.8978
QLoRA CodeBERT (ours)	**0.7526±0.003**	**0.6410**	**0.9113**	**0.8663±0.004**

QLoRA-CodeBERT achieves the highest F1 (0.7526) and recall (91.13%), establishing it as the primary ML backend. The calibrated ensemble achieves the highest ROC-AUC (0.9060), reflecting superior discriminative ability across all threshold settings — an advantage in ranking-based vulnerability prioritization.

D. Ensemble Ablation Study

Table VII quantifies the contribution of individual ensemble members. While LightGBM-Balanced achieves equal F1 to the full ensemble, the ensemble's substantially higher ROC-AUC (0.9050 vs. individual estimates of 0.85–0.87) reflects improved calibration and robustness.

TABLE VII
ENSEMBLE ABLATION — INDIVIDUAL VS. COMBINED (n=33,840 EVAL. SUBSET)

Model	F1	P	R	AUC
XGBoost-Conservative	0.6054	0.5122	0.7401	0.874
XGBoost-Aggressive	0.5999	0.5963	0.6036	0.862
LightGBM-Balanced	0.6079	0.5194	0.7327	0.871
CatBoost	0.5651	0.4367	0.8004	0.851
Ensemble (all four)	**0.6079**	0.5289	0.7146	**0.9050**

E. Two-Stage Gated Ensemble Analysis

Table VIII reports the precision-recall trade-off of the gated architecture. The 1.3-point absolute reduction in FPR (7.3% $\rightarrow$ 6.0%) may be preferred in deployment environments where developer triage time per warning is expensive.

TABLE VIII
TWO-STAGE GATED ENSEMBLE — DETAILED ANALYSIS (n=33,840)

Configuration	F1	P	R	FPR
Stage 1 Standalone	0.6139	0.5706	0.6642	7.3%
Stage 2 Standalone	0.5576	0.5172	0.6048	8.3%
Gated Ensemble	0.5895	**0.5892**	0.5897	**6.0%**

F. QLoRA Threshold Sensitivity

Table IX demonstrates the utility of threshold control. At $\tau = 0.30$, 14,361 of 14,444 vulnerable validation samples are caught (only 83 missed), which is practical for security-critical pre-commit screening.

TABLE IX
QLoRA Threshold Sensitivity on Validation Set (n=34,233)

Threshold τ	F1	P	R	Use Case
0.30	0.620	0.481	0.994	Security-critical
0.55	**0.753**	**0.654**	**0.886**	Production default
0.70	0.671	0.791	0.582	Precision-focused

G. Confusion Matrix Analysis (QLoRA, τ=0.55)

From the validation confusion matrix (n=34,233):

$$\text{TP} = 13{,}163, \quad \text{FN} = 1{,}281, \quad \text{FP} \approx 7{,}388, \quad \text{FPR} \approx 16.1\% \tag{17}$$

The model achieves 91.13% recall with 64.1% precision. In a security triage workflow — where a false negative (missed CVE shipped to production) is considerably more costly than a false positive (developer spends 3–5 minutes reviewing a safe function) — this operating point is considered acceptable. The independent static analysis layer provides a higher-precision corroborating signal for triaging ML flags.

H. Focal Loss Ablation

To validate the focal loss hyperparameter choice, we compare $\gamma \in \{1.0, 1.5, 2.0, 2.5\}$ on the validation set with fixed $\alpha = 0.75$:

TABLE X
Focal Loss Ablation — γ Search on Validation Set

γ	F1	Recall	Precision
1.0	0.718	0.874	0.611
1.5	0.739	0.893	0.630
2.0	**0.753**	**0.886**	**0.654**
2.5	0.741	0.871	0.644

$\gamma = 2.0$ (the original RetinaNet default [8]) achieves the best F1, confirming it as the appropriate focusing strength for this class ratio.

IX. Discussion

A. QLoRA as a Practical Paradigm

Our results confirm that QLoRA enables competitive vulnerability detection without high-end GPU hardware. The gap between QLoRA (F1: 0.7526) and the four-model ensemble (F1: 0.6143) is 13.8 F1 points, while the ensemble retains a 3.97-point ROC-AUC advantage (0.9060 vs. 0.8663). This suggests that for ranking/triage scenarios the ensemble remains valuable, while for pass/fail classification QLoRA is clearly preferable. Notably, our QLoRA result substantially exceeds the best published F1 on each constituent dataset individually (CodeBERT FT: 0.3785 on DiverseVul [10]; LineVul: 0.195 on MegaVul [12]), which we attribute to the substantially larger and more diverse training corpus, focal loss, and the extended 512-token input window enabled by QLoRA.

B. Why Results Exceed Single-Dataset Baselines

Prior baselines are trained and evaluated on individual, smaller datasets — the DiverseVul paper trains on a ∼331K union, MegaVul baselines train on ∼391K functions, and both collapse in cross-project evaluation. LLM-VUL trains on 718K deduplicated functions spanning all three sources simultaneously, providing diversity that improves generalisation. This cross-dataset advantage should be clearly distinguished from the comparison: our numbers are not directly comparable as they use a different train/test partition from all prior work.

C. Independent Results Architecture

A key design decision is to *never* merge static analysis and ML results through OR or AND logic. Merging creates opacity: when a function is flagged by both a tool and the ML model, it is unclear whether one would have sufficed. The current independent presentation allows security engineers to reason about inter-tool agreement explicitly, supporting a richer audit trail. The FPR of 16.1% at τ=0.55 is substantively mitigated by this design: a false positive from the ML model that is not corroborated by any static tool is a strong indicator for human review.

D. Limitations

Language coverage: The training corpus is 99.7% C code, limiting generalisation to C++, Java, Python, and other languages. Multi-language extension will require a polyglot architecture such as UniXcoder [33] or CodeT5+ [34].

Binary labels only: The ML model outputs binary predictions. CWE type assignment currently relies on static tool output. A future multi-head architecture would jointly predict binary label and CWE type from the same forward pass.

Precision ceiling: QLoRA's precision of 64.1% means roughly 36% of ML-flagged functions are false positives. A lightweight post-processing re-ranker (e.g., trained on a development-set FP signal) could reduce this substantially.

Inference deployment: QLoRA inference runs at full fp32 precision after adapter merging, requiring more VRAM than training. TorchScript or ONNX export would improve throughput for latency-sensitive deployments.

Dataset bias: All training data originates from open-source C projects. The "unseen projects" collapse documented by Chen et al. [10] — where CodeBERT F1 drops from 37.85% to 11.94% on held-out repositories — is likely mitigated but not eliminated by our larger corpus.

E. Future Work

Planned extensions include: (1) multi-language support via CodeT5+ or UniXcoder fine-tuning; (2) multi-head classification for joint binary and CWE-type prediction; (3) integration of AST/CFG/PDG graph representations as additional ensemble features; (4) IDE plugin development for real-time inline feedback; and (5) RLHF-based adaptive learning from developer feedback on false positives.

X. Conclusion

We have presented LLM-VUL, a hybrid vulnerability detection platform combining a QLoRA-fine-tuned CodeBERT adapter, a 304-feature gradient boosting ensemble, and an independent static analysis layer within a production-ready web application. On a deduplicated unified corpus of 718,813 labeled C/C++ functions, our QLoRA model achieves F1 = 0.7526 ($\pm$0.003), recall = 91.13%, and ROC-AUC = 0.8663 — trained within a 6 GB VRAM budget by updating only 1.77M parameters (1.4% of the base model). Focal loss with $\gamma = 2.0$ and $\alpha = 0.75$, validated by grid search, is shown to be an effective strategy for the 11.7:1 class imbalance inherent in realistic vulnerability datasets. The two-stage gated ensemble reduces false positive rate to 6.0% for precision-critical workflows, while the QLoRA backend maximizes recall for security-critical screening. Compared to the best published transformer baselines on the constituent datasets (CodeBERT FT: F1=0.3785 on DiverseVul; LineVul: F1=0.195 on MegaVul), the combined effect of a larger training corpus, focal loss, and 512-token input represents a substantial advance in the function-level vulnerability detection task. The platform is fully operational as a web service with asynchronous job management, GitHub repository-level scanning, and a React/TypeScript frontend. All training configurations, feature engineering code, and model artifacts are released at https://github.com/debashish17/LLM-VUL to support reproducibility and further research in AI-assisted software security.

Acknowledgments. The authors thank the maintainers of DiverseVul, MegaVul, and Devign for making their datasets publicly available.

References

[1] NIST, "National Vulnerability Database (NVD) Statistics," *National Institute of Standards and Technology*, 2023. [Online]. Available: https://nvd.nist.gov/general/nvd-dashboard

[2] MITRE Corporation, "2023 CWE Top 25 Most Dangerous Software Weaknesses," *MITRE CWE Program*, 2023. [Online]. Available: https://cwe.mitre.org/top25/archive/2023/2023_top25_list.html

[3] B. Chess and G. McGraw, "Static analysis for security," *IEEE Security & Privacy*, vol. 2, no. 6, pp. 76–79, Nov.–Dec. 2004.

[4] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A pre-trained model for programming and natural languages," in *Proc. EMNLP Findings*, 2020, pp. 1536–1547.

[5] D. Guo *et al.*, "GraphCodeBERT: Pre-training code representations with data flow," in *Proc. ICLR*, 2021.

[6] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. ICLR*, 2022.

[7] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Proc. NeurIPS*, 2023, pp. 10088–10115.

[8] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. ICCV*, 2017, pp. 2980–2988.

[9] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," in *Proc. NeurIPS*, 2019, pp. 10197–10207.

[10] Y. Chen, Z. Ding, L. Alsmadi, Z. Wu, G. Hu, Y. Guo, and M. A. Al-Shabi, "DiverseVul: A new vulnerable source code dataset for deep learning based vulnerability detection," in *Proc. ACM RAID*, 2023.

[11] Y. Ni, S. Wang, X. Zhang, X. Dou, and T. Liu, "MegaVul: A C/C++ vulnerability dataset with comprehensive code representation," in *Proc. MSR*, 2023, pp. 7–11.

[12] Y. Ni, S. Shen, Z. Xu, H. Yin, and S. Wang, "Learning-based models for vulnerability detection: An extensive study," *arXiv preprint arXiv:2408.07526*, 2024.

[13] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, and Y. Zhong, "VulDeePecker: A deep learning-based system for vulnerability detection," in *Proc. NDSS*, 2018.

[14] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, and Z. Chen, "SySeVR: A framework for using deep learning to detect software vulnerabilities," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2244–2258, 2022.

[15] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet," *IEEE Trans. Softw. Eng.*, vol. 48, no. 9, pp. 3280–3296, 2022.

[16] M. Fu and C. Tantithamthavorn, "LineVul: A transformer-based line-level vulnerability prediction," in *Proc. MSR*, 2022, pp. 608–620.

[17] H. Hanif and S. Maffeis, "VulBERTa: Simplified source code pre-training for vulnerability detection," in *Proc. IJCNN*, 2022.

[18] V.-H. Nguyen and L.-T. Tran, "ReGVD: Revisiting graph neural networks for vulnerability detection," in *Proc. ICSE Companion*, 2022, pp. 235–239.

[19] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. ACM KDD*, 2016, pp. 785–794.

[20] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Proc. NeurIPS*, 2017, pp. 3146–3154.

[21] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in *Proc. NeurIPS*, 2018, pp. 6638–6648.

[22] S. Neuhaus, T. Zimmermann, C. Holler, and A. Zeller, "Predicting vulnerable software components," in *Proc. ACM CCS*, 2007, pp. 529–540.

[23] Y. Shin and L. Williams, "An empirical model to predict security vulnerabilities using code complexity metrics," in *Proc. ESEM*, 2008, pp. 315–317.

[24] Y. Li, S. Wang, and T. N. Nguyen, "Vulnerability detection with fine-grained interpretations," in *Proc. ESEC/FSE*, 2021, pp. 292–303.

[25] R. Russell, L. Kim, L. Hamilton, T. Lazovich, J. Harer, O. Ozdemir, P. Ellingwood, and M. McConley, "Automated vulnerability detection in source code using deep representation learning," in *Proc. IEEE ICMLA*, 2018, pp. 757–762.

[26] D. A. Wheeler, "Flawfinder," 2023. [Online]. Available: https://dwheeler.com/flawfinder/

[27] D. Marjamäki, "Cppcheck: A tool for static C/C++ code analysis," *SourceForge*, 2023. [Online]. Available: https://cppcheck.sourceforge.io

[28] Semgrep, Inc., "Semgrep: Lightweight static analysis for many languages," 2023. [Online]. Available: https://semgrep.dev

[29] A. Bessey *et al.*, "A few billion lines of code later: Using static analysis to find bugs in the real world," *Commun. ACM*, vol. 53, no. 2, pp. 66–75, 2010.

[30] D. Evans and D. Larochelle, "Improving security using extensible lightweight static analysis," *IEEE Softw.*, vol. 19, no. 1, pp. 42–51, 2002.

[31] J. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," *Advances in Large Margin Classifiers*, vol. 10, no. 3, pp. 61–74, 1999.

[32] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," in *Proc. NeurIPS*, 2022.

[33] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, "UniXcoder: Unified cross-modal pre-training for code representation," in *Proc. ACL*, 2022, pp. 7212–7225.

[34] Y. Wang, H. Le, A. D. Gotmare, N. D. Q. Bui, J. Li, and S. C. H. Hoi, "CodeT5+: Open code large language models for code understanding and generation," in *Proc. EMNLP*, 2023, pp. 1069–1088.